

Rodrigo Fernandez

Senior Ruby on Rails / React Full Stack Engineer

Katy, Texas | +1 (407) 801-1287 | www.linkedin.com/in/rodrigobermejo-fernandez-9a5a0664 | rodrigobfdev@gmail.com

Summary

Senior **Ruby on Rails 5.2–7.1 / React 16–18 Full Stack Engineer** with strong experience in cloud hosting, fintech, healthcare, SaaS, and eCommerce industries, building secure APIs, multi-tenant platforms, admin portals, billing workflows, and high-volume integrations using **Ruby 2.6–3.3, Rails 5.2–7.1, PostgreSQL 11–15, Redis 5–7, Sidekiq 6–7, RSpec 3.x**, Docker, AWS, and CI/CD, with a practical record of fixing production bugs, reducing slow query latency, upgrading legacy Rails services, improving test coverage, and delivering business features that increased reliability, deployment confidence, customer trust, and team velocity.

Skills

- **Backend:** **Ruby 2.6–3.3, Ruby on Rails 5.2–7.1**, Rails API, ActiveRecord, ActionCable, Sidekiq, Redis, RSpec, Minitest, GraphQL, REST API, OAuth2, JWT, Devise, Pundit, CanCanCan
- **Frontend:** **React 16–18**, TypeScript 4.x–5.x, JavaScript ES6+, Redux Toolkit, React Query, Next.js, HTML5, CSS3, Tailwind CSS, Bootstrap
- **Database:** **PostgreSQL 11–15**, MySQL 5.7–8.x, Redis 5–7, Elasticsearch 7.x–8.x, query optimization, indexing, database migration, schema refactoring
- **Cloud & DevOps:** AWS EC2, S3, RDS, ECS, Lambda, CloudWatch, Docker, Kubernetes, GitHub Actions, GitLab CI, Jenkins, Nginx, Linux
- **Testing & Quality:** RSpec 3.x, Capybara, FactoryBot, Jest, Cypress, RuboCop, Brakeman, SimpleCov, code review, CI quality gates
- **Architecture:** Monolith modernization, Rails API architecture, service objects, background jobs, event-driven workflows, multi-tenant SaaS, payment integration

Experience

Lightedge — Senior Software Engineer (*Apr 2020 – Mar 2026*)

- Led **Ruby on Rails 6.0–7.1** modernization for cloud hosting customer portals, improving account provisioning, billing visibility, and service management across enterprise users.
- Migrated legacy Rails modules from **Rails 5.2 to Rails 7.1** with Zeitwerk, Bundler updates, and dependency cleanup, reducing deployment rollback risk by **35%**.
- Resolved a production memory leak in **Sidekiq 6–7** workers by batching records, tuning Redis retries, and reducing long-running job failures by **42%**.
- Improved slow **PostgreSQL 12–15** reporting queries with composite indexes, query plan analysis, and ActiveRecord refactoring, cutting dashboard load time by **58%**.
- Built secure REST APIs using **Rails API**, JWT authentication, Pundit policies, and request specs for customer infrastructure workflows and role-based permissions.
- Implemented billing reconciliation features with background jobs, idempotent service objects, and audit logging, reducing manual finance corrections by **31%**.
- Fixed intermittent payment webhook duplication by adding idempotency keys, transaction locks, and retry-safe Sidekiq logic across external provider callbacks.

- Refactored large Rails controllers into service objects, query objects, and serializers, making business logic easier to test and improving review speed by **24%**.
- Integrated **React 17–18** admin screens with Rails JSON APIs, improving customer support workflows for provisioning, invoice review, and account-level activity tracking.
- Expanded **RSpec 3.x** coverage for billing, permissions, and infrastructure APIs, raising backend test coverage from **64% to 86%** without slowing CI pipelines.
- Supported AWS-based deployments with Docker, GitHub Actions, RDS, S3, CloudWatch, and rollback scripts to improve production release confidence.
- Collaborated with product, QA, DevOps, and support teams to convert recurring customer issues into stable Rails features, monitoring alerts, and reusable internal tools.

NIX United — Software Engineer *(Oct 2016 – Mar 2020)*

- Developed **Ruby on Rails 5.2–6.1** SaaS and fintech applications using PostgreSQL, Redis, Sidekiq, React, and third-party payment integrations.
- Upgraded multiple client applications from **Ruby 2.5 to Ruby 3.0** by fixing gem conflicts, keyword argument issues, and CI failures during staged releases.
- Resolved N+1 query issues in ActiveRecord associations using eager loading, database indexes, and pagination, improving API response time by **47%**.
- Built subscription, invoicing, and user-management features with Devise, Stripe APIs, service objects, and background jobs for recurring billing workflows.
- Fixed race conditions in payment processing by adding database constraints, transaction boundaries, and retry-safe jobs, reducing duplicate charge issues by **39%**.
- Implemented GraphQL and REST endpoints for customer dashboards, keeping response contracts stable while frontend teams migrated screens into **React 16–17**.
- Created reusable RSpec factories, request specs, and contract tests that helped reduce regression bugs by **28%** across recurring client releases.
- Improved Rails deployment stability by updating Docker images, environment variables, asset compilation, and GitLab CI pipeline stages.
- Refactored legacy ERB-heavy pages into React components where needed, while keeping Rails views for simpler admin workflows and faster delivery.
- Supported production incidents by tracing logs, reviewing Sidekiq retries, checking PostgreSQL locks, and applying hotfixes with safe rollback plans.
- Worked flexibly across backend APIs, frontend fixes, database scripts, and DevOps support to keep client milestones moving without blocking other teams.

Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Built eCommerce and digital service platforms with **Ruby on Rails 4.2–5.2**, PostgreSQL, jQuery, Bootstrap, Redis, and background job processing.
- Migrated legacy Rails applications from **Rails 4.2 to Rails 5.2** by replacing deprecated callbacks, updating gems, and correcting strong parameter issues.
- Fixed cart pricing bugs caused by tax rounding and coupon stacking logic, improving checkout accuracy and reducing support tickets by **33%**.
- Implemented product catalog, order management, inventory, and admin reporting features using ActiveRecord models, service classes, and scheduled workers.
- Improved page performance by optimizing database queries, compressing assets, and caching repeated fragments, reducing average page load time by **41%**.
- Integrated payment, shipping, and email provider APIs with retry handling, error logging, and admin tools for recovering failed transactions.

- Added RSpec model and request tests around checkout, user registration, and order updates, increasing release confidence for high-traffic campaigns.
- Resolved production errors from malformed customer inputs by adding validations, sanitization, and clearer exception handling in Rails forms.
- Supported frontend delivery with **JavaScript ES6**, jQuery, Bootstrap, and responsive layouts while coordinating closely with designers and QA.
- Documented deployment steps, rollback notes, and common Rails issue fixes so junior developers could support releases more independently.

Aurio — Junior Software Developer *(Oct 2012 – Dec 2013)*

- Supported internal business applications using **Ruby on Rails 3.2–4.2**, MySQL, PostgreSQL, JavaScript, jQuery, HTML, CSS, and basic Linux deployment workflows.
- Built CRUD modules for customer records, admin approvals, reporting screens, and notification workflows under senior engineer guidance.
- Fixed validation and form submission bugs in Rails controllers and models, reducing repeated user-entry errors by **26%** across internal teams.
- Assisted with migration from older Rails patterns to cleaner MVC structure by moving duplicated logic into helpers, concerns, and model methods.
- Improved SQL queries and ActiveRecord scopes for reports, helping reduce timeout issues in large customer exports by **22%**.
- Added basic RSpec tests for models, controllers, and service logic, creating safer coverage around frequently changed business rules.
- Supported bug triage by reproducing user issues, reading Rails logs, checking database records, and preparing clear notes for senior developers.
- Updated UI pages with jQuery, Bootstrap, and CSS fixes to improve usability while keeping compatibility with the existing Rails application.
- Learned production support practices through small hotfixes, code reviews, deployment checklists, and careful communication with QA and stakeholders.

Microsoft — Software Developer Intern *(Jun 2012 – Sep 2012)*

- Supported internal engineering tools using **C#, .NET Framework 4.0–4.5, ASP.NET MVC 3–4**, JavaScript, jQuery, and SQL Server, gaining practical experience in enterprise software quality and structured development workflows.
- Assisted senior engineers with debugging UI defects, validation issues, and data-handling problems, documenting root causes clearly so fixes could be reviewed and released with lower regression risk.
- Built small internal web modules for tracking records, reviewing status changes, and improving team visibility, applying clean HTML, CSS, JavaScript, and backend integration patterns.
- Resolved early-stage bugs related to inconsistent form validation and SQL query results by reproducing issues locally, checking logs, and working with mentors to apply safer input-handling rules.

Education

Texas State University

Bachelor's Degree in Computer Science *(Sep 2008 – May 2012)*